燕山大学课程设计报告

课程名称：计算机组成原理

题目：复杂模型机快速幂算法实现

班　　级：计算机 23-5 班
组　　号：第 4 组
课题负责人：李济岑
课题组成员：

姓名	学号	自评成绩	分工
李济岑	202211090252	100	主要代码编写
孔捷	202311040127	100	测试和报告
那圣奇	202311040130	100	实验箱连线

课题开发日期：2025 年 12 月
指导教师：　邹晓红 孔维航 杜国栋 张秀杰 任大伟 许莹

目录

目录 **1**

1 前言 **2**

1.1 项目背景 2
1.2 目的意义 2

2 主要任务 **3**

2.1 项目概述 3
2.2 需求分析 4

3 算法设计 **4**

3.1 快速幂原理及算法 4
3.2 快速乘法原理及实现 5
3.3 汇编转换机器码 5

4 复杂模型机 **6**

4.1 数据格式 6
4.2 指令设计 6
4.3 指令格式 7
4.4 指令系统 9
4.5 微指令设计 10

5 项目内容 **14**

5.1 实施方案 14
5.2 实验线路设计 14
5.3 快速幂流程 15
5.4 快乘流程 17

6 项目成果 **19**

6.1 线路连接 19
6.2 测试样例 19
6.3 完整代码 20

7 总结 **20**

参考文献： **21**

附录 A **21**

附录 B **24**

附录 C **32**

摘要：在本实验中，我们设计和实现了一个复杂模型机来执行结合快速乘法实现快速幂的操作。通过该实验，我们深入研究了快速幂的原理及其与快速乘法的结合应用，并将其应用于硬件电路设计中。我们进行了详细的设计过程和实验结果分析，展示了该复杂模型机在执行快速幂运算时的性能和准确性。

关键词：复杂模型机；快速幂；计算机组成原理；汇编指令；

1 前言

1.1 项目背景

本课程研究项目是《计算机组成原理》课程学习的一个重要组成部分。本项目聚焦于通过快速乘法实现快速幂的计算机组成原理实践，通过课程研究项目的实施，使学生在掌握计算机组成课程理论的基础上，结合复杂模型机的设计与实现。

本项目深入了解并掌握组成计算机各主要功能部件的相关知识及工作原理，掌握研究（开发）技术报告的撰写方法，使学生具备独立设计计算机的基本能力，提高综合应用已有知识解决实际问题的能力，更好地培养计算机专业学生的专业技术能力和综合素质。

1.2 目的意义

本项目以《计算机组成原理》课程实验为载体，围绕“复杂模型机结合快速乘法实现快速幂”的目标开展设计与实现工作。通过将快速幂算法落到具体指令序列、机器码以及硬件连线与调试流程中，训练从“算法—指令—微操作—硬件执行”的完整工程思维链路。

项目的具体意义主要体现在：

1. **巩固组成原理核心知识**：在复杂模型机框架下，理解并掌握寄存器、存储器、控制逻辑等部件的协同工作机制，对计算机组成原理有更深刻认识。

2. **提升软硬件协同能力**：开发过程包含代码编写、算法实现、硬件电路设计与测试等关键环节，锻炼了学生动手实践能力。

3. **强化工程化与文档能力**：按“选题—框架—程序设计—编码实现—运作调试”的流程组织工作，形成可复现实验资料与报告。

2 主要任务

2.1 项目概述

本项目实现一个可运行的复杂模型机程序，通过**快速乘法**支撑**快速幂**计算，并在 CMA 平台中完成加载、运行与结果验证。整体工作包括：算法流程设计、指令序列规划、机器码生成、与实验箱硬件连接后进行联调，最终展示运行结果与正确性分析。

项目实现目标可概括为：

1. 输入底数与指数（实验数据按约定地址写入）；
2. 使用“二分思想”的快速幂流程逐位处理指数（按位判断奇偶、必要时乘入结果、底数自乘、指数缩小）；
3. 乘法运算由快速乘法程序完成，并将最终结果按约定写回内存验证使用。

2.2 需求分析

功能需求：支持快速幂运算流程，底层需支持快速乘法（模型机无乘法指令时的替代实现），以快速乘法实现快速幂算法。

指令与控制需求：需要定义适配的指令集与控制逻辑，覆盖指令解码、执行、存储器访问与快乘支撑，编写指令集转换程序以汇编得到指令集。

实验验证需求：能在 CMA 软件中观察周期级指令操作与数据流向，并在实验箱连线运行验证结果，得出快速幂的实现效果展示。

3 算法设计

3.1 快速幂原理及算法

快速幂的核心思想是将指数(b)表示为其二进制形式，并将(a^b)分解为若干个小的任务。例如，如果(b)有(k)个二进制位，那么我们可以通过计算(k-1)次乘法来得到(a^b)。

$$n = n_t 2^t + n_{t-1} 2^{t-1} + n_{t-2} 2^{t-2} + \cdots + n_1 2^1 + n_0 2^0$$

那么就有：

$$a^n = \left(a^{n_t 2^t + \cdots + n_0 2^0}\right)$$

$$= a^{n_0 2^0} \times a^{n_1 2^1} \times \cdots \times a^{n_t 2^t}$$

可以将原本的幂运算转换为形式相同的子问题的乘积，时间复杂度从 O(n)优化为O(logn)。以下是快速幂的 Python 代码演示：

```
def pow_fast(base: int, power: int) -> int:
      ret = 1
      while power:
                  if power % 2 == 1:
                  ret *= base
                  base *= base
                  power //= 2
      return ret
```

3.2 快速乘法原理及实现

由于模型机并未提供乘法指令，故通过移位、加法指令，实现了快速乘法来优化快速幂 ，同时又能满足乘法操作。

当计算 M*N%P 时，为了避免溢出，我们常常采用的方法是利用分配律将式子转换为：（ M % P ）*（ N % P ）% P，因为一 个值模 P 后 其 范 围为 0 到 P-1，只要 P*P 不超过数据上限即可。

由于模型机并未提供取余操作，故可以用如下公 式来满足快速乘法的取余当 P 为 2 的任意次方时，A%P = A&（ P-1）就像快速幂本 质上是乘法，并利用二进制思想减少运算次数。

快速 乘 法 本质上是 加法，并利用了二进制思想来 减少运算次数。因为 本 质在做加法运算，所以只要 P+P 不超过数据上限即可

以下是快速乘法的 Python 代码演示：

```
def quickmul(a: int, b: int) -> int:
      ret = 0
      while b:
            if b & 1:
                  ret += a
            a <<= 1
            b >>= 1
      return ret
```

3.3 汇编转换机器码

因难以直接用机器码进行编程，通常是先编写汇编语言，再通过后文复杂模型机的映射表，对 15 个机器指令和 54 个机器码进行查表到对应机器码，本项目编写了 Python 脚本程序，用于将汇编语言翻译成机器码 txt 文件。

打表部分程序如下，使用 python script.py asm.txt pm.txt 进行翻译：

```
OP = {
    "ADD": 0x0, "AND": 0x1,   "IN": 0x2,  "OUT": 0x3,   "STA": 0xD,
    "MOV": 0x4, "HALT": 0x5,  "LDI": 0x6, "INC": 0x7,   "JMP": 0xE,
    "SUB": 0x8, "OR": 0x9,    "RR": 0xA,  "LAD": 0xC,   "BZC": 0xF,
}
DEFAULT_MICRO: Dict[int, str] = {
    0x00: "000001",    0x01: "006D43",    0x03: "107070",
    0x04: "002405",    0x05: "04B201",    0x06: "002407",
    0x07: "013201",    0x08: "106009",    0x09: "183001",
```

```
    0x0A: "106010",     0x0B: "000001",     0x0C: "103001",
    0x0D: "200601",     0x0E: "005341",     0x0F: "0000CB",
    0x10: "280401",     0x11: "103001",     0x12: "06B201",
    0x13: "002414",     0x14: "05B201",     0x15: "002416",
    0x16: "01B201",     0x17: "002418",     0x18: "02B201",
    0x1B: "005341",     0x1C: "10101D",     0x1D: "10608C",
    0x1E: "10601F",     0x1F: "101020",     0x20: "10608C",
    0x28: "101029",     0x29: "00282A",     0x2A: "04E22B",
    0x2B: "04928C",     0x2C: "10102D",     0x2D: "002C2E",
    0x2E: "04E22F",     0x2F: "04928C",     0x30: "001604",
    0x31: "001606",     0x32: "006D48",     0x33: "006D4A",
    0x34: "003401",     0x35: "000035",     0x36: "006D51",
    0x37: "001612",     0x38: "001613",     0x39: "001615",
    0x3A: "001617",     0x3B: "000001",     0x3C: "006D5C",
    0x3D: "006D5E",     0x3E: "006D68",     0x3F: "006D6C",
}
```

4 复杂模型机

4.1 数据格式

模型机规定采用定点补码表示法表示数据，字长为 8 位，8 位全用来表示数据（最高位不表示符号），数值表示范围是： $0 \leqslant X \leqslant 2^8 - 1$。

4.2 指令设计

模型机设计三大类指令共十五条，其中包括运算类指令、控制转移类指令，数据传送类指令。运算类指令包含三种运算，算术运算、逻辑运算和移位运算，设计有 6 条运算类指令，分别为：ADD、AND、INC、SUB、OR、RR，所有运算类指令都为单字节，寻址方式采用寄存器直接寻址。控制转移类指令有三条 HLT、JMP、BZC，用以控制程序的分支和转移，其中 HLT 为单字节指令，JMP 和 BZC 为双字节指令。数据传送类指令有 IN、OUT、MOV、LDI、LAD、STA 共 6 条，用以完成寄存器和寄存器、寄存器和 I/O、寄存器和存储器之间的数据交换，除 MOV 指令为单字节指令外，其余均为双字节指令。

4.3 指令格式

所有单字节指令（ADD、AND、INC、SUB、OR、RR、HLT 和 MOV）格式如下：

7 6 5 4	3 2	1 0
OP-CODE	RS	RD

其中，OP-CODE 为操作码，RS 为源寄存器，RD 为目的寄存器，并规定：

RS 或 RD	选定的寄存器
00	R0
01	R1
10	R2
11	R3

IN 和 OUT 的指令格式为：

7 6 5 4 （1）	3 2 （1）	1 0 （1）	7—0（2）
OP-CODE	RS	RD	P

其中括号中的 1 表示指令的第一字节，2 表示指令的第二字节，OP-CODE 为操作码，RS 为源寄存器，RD 为目的寄存器，P 为 I/O 端口号，占用一个字节，系统的 I/O 地址译码原理见图 6-1（在地址总线单元）。

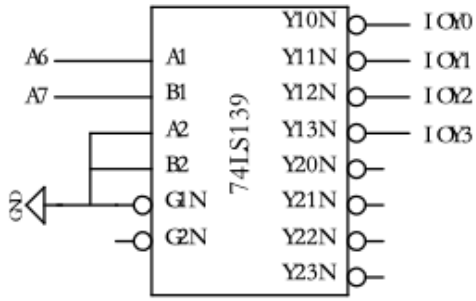


图 4-1　I/O 地址译码原理图

由于用的是地址总线的高两位进行译码，I/O 地址空间被分为四个区，如表 6-1 所示：

表 4-1　I/O 地址空间分配

A7　A6	选定	地址空间
00	IOY0	00-3F
01	IOY1	40-7F
10	IOY2	80-BF
11	IOY3	C0-FF

系统设计五种数据寻址方式，即立即、直接、间接、变址和相对寻址，LDI 指令为立即寻址，LAD、STA、JMP 和 BZC 指令均具备直接、间接、变址和相对寻址能力。

LDI 的指令格式如下，第一字节同前一样，第二字节为立即数。

7 6 5 4 （1）	3 2 （1）	1 0 （1）	7—0（2）
OP-CODE	RS	RD	data

LAD、STA、JMP 和 BZC 指令格式如下。

7 6 5 4 （1）	3 2 （1）	1 0 （1）	7—0（2）
OP-CODE	M	RD	D

其中 M 为寻址模式，具体见表 4-2，以 R2 做为变址寄存器 RI。

表 4-2　寻址方式

寻址模式 M	有效地址 E	说　明
00	E = D	直接寻址
01	E =（D）	间接寻址
10	E =（RI）+ D	RI 变址寻址

11	E =（PC）+ D	相对寻址

4.4 指令系统

本模型机共有 15 条基本指令，表 4-3 列出了各条指令的格式、汇编符号、指令功能。

表 4-3　指令描述

助记符号	指令格式				指令功能
MOV RD, RS	0100	RS	RD		RS → RD
ADD RD, RS	0000	RS	RD		RD + RS → RD
SUB RD, RS	1000	RS	RD		RD - RS → RD
AND RD, RS	0001	RS	RD		RD∧RS → RD
OR RD, RS	1001	RS	RD		RD∨RS → RD
RR RD, RS	1010	RS	RD		RS右环移 → RD
INC RD	0111	**	RD		RD+1 → RD
LAD M D, RD	1100	M	RD	D	E → RD
STA M D, RS	1101	M	RD	D	RD → E
JMP M D	1110	M	**	D	E → PC
BZC M D	1111	M	**	D	当FC或FZ=1时，E → PC
IN RD, P	0010	**	RD	P	[P] → RD
OUT P, RS	0011	RS	**	P	RS → [P]
LDI RD, D	0110	**	RD	D	D → RD
HALT	0101	**	**		停机

4.5 微指令设计

设计微指令结构应该追求的目标是：

（1）有利于缩短微指令字长度 ；

（2）有利于减小控制存储器的容量；

（3）有利于提高微程序的执行速度；

（4）有利于对微指令的修改；

（5）有利于提高微程序设计的灵活性。

基于上述考虑，我们对于微指令的设计应该尽量减少冗余。计算机微指令存在于微程序控制器中，每一条机器指令都是由若干条微指令序列来完成。因此，微指令是设计的核心，只有将每一条微指令理解透彻，才能完成指令系统的设计以及程序的正确执行。

本文设计的微指令格式为 24 位 。本实验中要用到四个通用寄存器 R3…R0，而对寄存器的选择是通过指令的低四位，为此还得设计一个寄存器译码电路，在 IR 单元的 REG_DEC（GAL16V8）中实现，如图 4-2 所示。

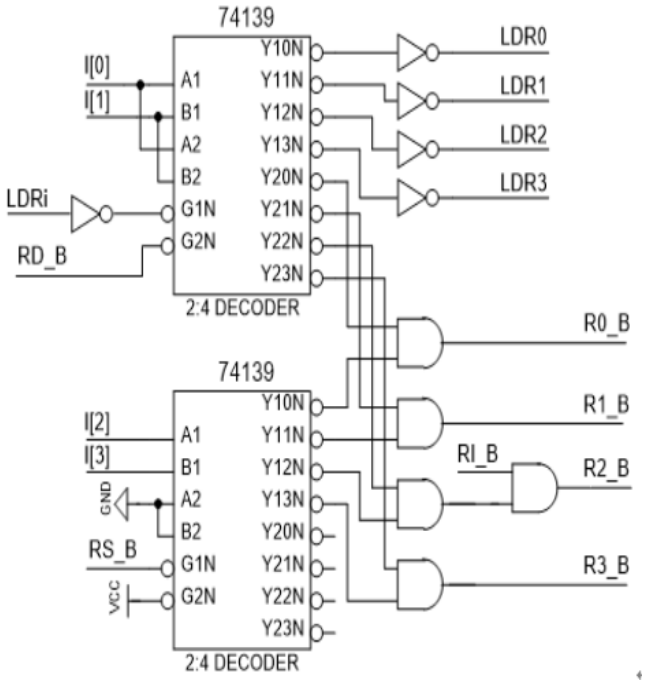


图 4-2 寄存器译码原理图

根据机器指令系统要求，设计微程序流程图及确定微地址，如图 4-3 所示。按照系统建议的微指令格式，见表 4-4，参照微指令流程图，将每条微指令代码化，译成二进制代码表，见表 4-5，并将二进制代码表转换为联机操作时的十六进制格式文件。

表 4-4 微指令格式

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	CN	WR	RD	ICM	S3-S0	A字段	B字段	C字段	UA5-UA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDRi
1	0	0	保留
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

B字段

11	10	9	选择
0	0	0	NOP
0	0	1	ALU_B
0	1	0	RS_B
0	1	1	RD_B
1	0	0	RI_B
1	0	1	保留
1	1	0	PC_B
1	1	1	保留

C字段

8	7	6	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	P<2>
0	1	1	P<3>
1	0	0	保留
1	0	1	LDPC
1	1	0	保留
1	1	1	保留

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	CN	WR	RD	ICM	S3-S0	A字段	B字段	C字段	UA5-UA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDRi
1	0	0	保留
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

B字段

11	10	9	选择
0	0	0	NOP
0	0	1	ALU_B
0	1	0	RS_B
0	1	1	RD_B
1	0	0	RI_B
1	0	1	保留
1	1	0	PC_B
1	1	1	保留

C字段

8	7	6	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	P<2>
0	1	1	P<3>
1	0	0	保留
1	0	1	LDPC
1	1	0	保留
1	1	1	保留

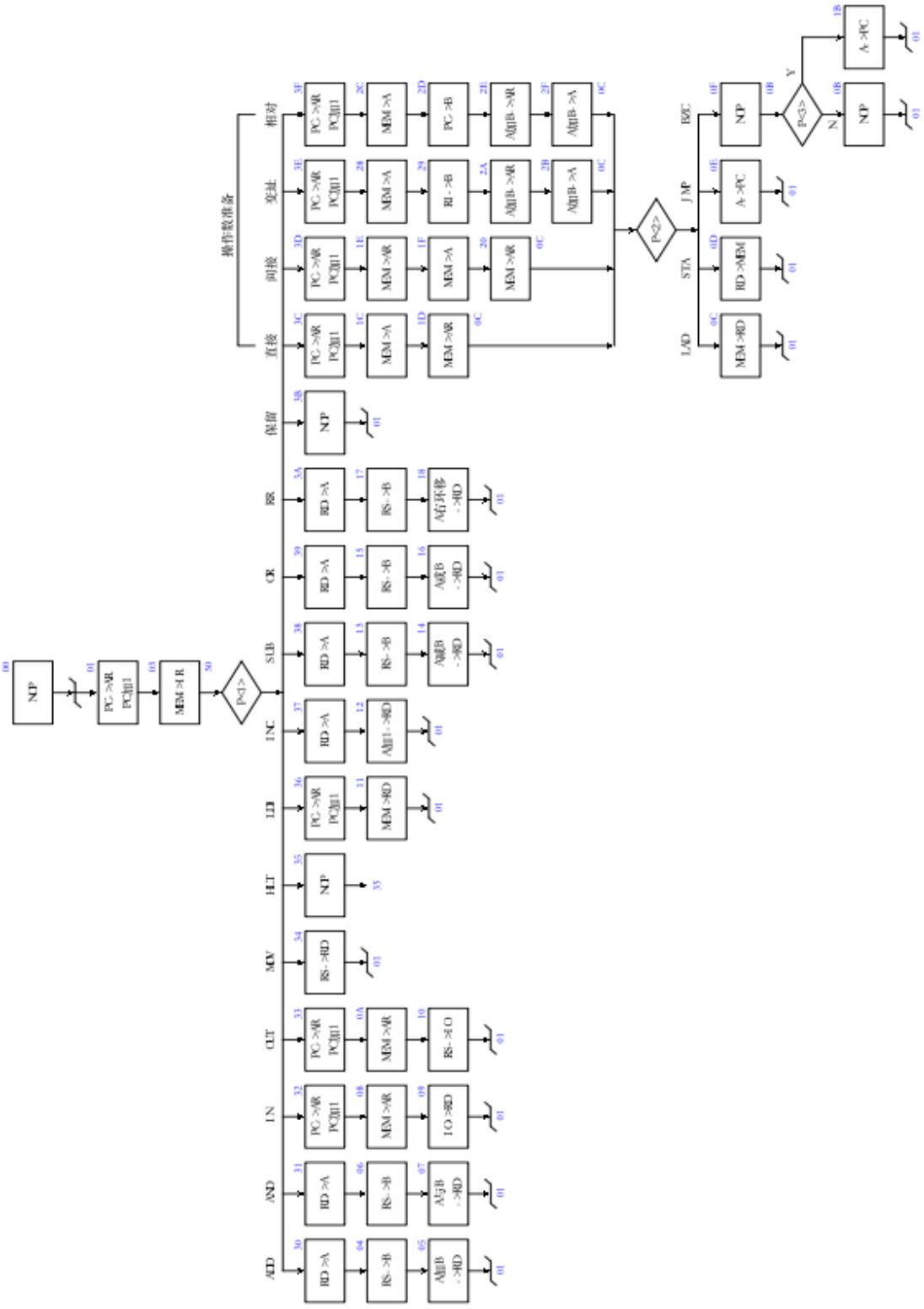

NOP
PC ->MR
PC加1
MEM ->IR
P<7>
操作数准备
ADD
AND
IN
OUT
MOV
HLT
LDI
INC
SUB
OR
RR
保留
直接
间接
变址
相对
LAD
STA
JMP
RSC
P<5>
Y
N
A ->PC
MDR ->MEM
MEM ->MDR
P<2>

图 4-3 微程序图

表 4-5 二进制代码表

地址	十六进制表示	高五位	S3-S0	A 字段	B 字段	C 字段	UA5-UA0
00	00 00 01	00000	0000	000	000	000	000001
01	00 6D 43	00000	0000	110	110	101	000011
03	10 70 70	00010	0000	111	000	001	110000
04	00 24 05	00000	0000	010	010	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
06	00 24 07	00000	0000	010	010	000	000111
07	01 32 01	00000	0010	011	001	000	000001
08	10 60 09	00010	0000	110	000	000	001001
09	18 30 01	00011	0000	011	000	000	000001
0A	10 60 10	00010	0000	110	000	000	010000
0B	00 00 01	00000	0000	000	000	000	000001
0C	10 30 01	00010	0000	011	000	000	000001
0D	20 06 01	00100	0000	000	011	000	000001
0E	00 53 41	00000	0000	101	001	101	000001
0F	00 00 CB	00000	0000	000	000	011	001011
10	28 04 01	00101	0000	000	010	000	000001
11	10 30 01	00010	0000	011	000	000	000001
12	06 B2 01	00000	1101	011	001	000	000001
13	00 24 14	00000	0000	010	010	000	010100
14	05 B2 01	00000	1011	011	001	000	000001
15	00 24 16	00000	0000	010	010	000	010110
16	01 B2 01	00000	0011	011	001	000	000001
17	00 24 18	00000	0000	010	010	000	011000
18	02 B2 01	00000	0101	011	001	000	000001
1B	00 53 41	00000	0000	101	001	101	000001
1C	10 10 1D	00010	0000	001	000	000	011101
1D	10 60 8C	00010	0000	110	000	010	001100
1E	10 60 1F	00010	0000	110	000	000	011111
1F	10 10 20	00010	0000	001	000	000	100000
20	10 60 8C	00010	0000	110	000	010	001100
28	10 10 29	00010	0000	001	000	000	101001
29	00 28 2A	00000	0000	010	100	000	101010
2A	04 E2 2B	00000	1001	110	001	000	101011
2B	04 92 8C	00000	1001	001	001	010	001100
2C	10 10 2D	00010	0000	001	000	000	101101
2D	00 2C 2E	00000	0000	010	110	000	101110
2E	04 E2 2F	00000	1001	110	001	000	101111
2F	04 92 8C	00000	1001	001	001	010	001100
30	00 16 04	00000	0000	001	011	000	000100
31	00 16 06	00000	0000	001	011	000	000110
32	00 6D 48	00000	0000	110	110	101	001000
33	00 6D 4A	00000	0000	110	110	101	001010
34	00 34 01	00000	0000	011	010	000	000001
35	00 00 35	00000	0000	000	000	000	110101
36	00 6D 51	00000	0000	110	110	101	010001
37	00 16 12	00000	0000	001	011	000	010010
38	00 16 13	00000	0000	001	011	000	010011

39	00 16 15	00000	0000	001	011	000	010101
3A	00 16 17	00000	0000	001	011	000	010111
3B	00 00 01	00000	0000	000	000	000	000001
3C	00 6D 5C	00000	0000	110	110	101	011100
3D	00 6D 5E	00000	0000	110	110	101	011110
3E	00 6D 68	00000	0000	110	110	101	101000
3F	00 6D 6C	00000	0000	110	110	101	101100

5 项目内容

5.1 实施方案

项目分为确定选题、构思框架结构、完成程序设计、编码实现 、运作调试 5 个阶段。

（1）确定选题：对复杂模型机以及给定的指令与寄存器进行分析，本组决定设计并实现快速乘法与快速幂。

（2）构思框架结构：确定项目要实现的目标，确定程序框架结构 。

（3）完成程序设计：完成程序的各个模块及跳转的设计。

（4）编码实现： 通过编程在复杂模型机上完成上述设计的代码并翻译为机器码 。

（5）运作调试：在设计的复杂模型机上运行程序，根据 CMA 上的模型运行情 况判断代码的正确性，根据相应结果对程序进行调试 。

5.2 实验线路设计

按照给定连接图对实验箱进行连线，完成复杂模型机硬件运行环境搭建，为后续机器码装载与运行结果验证提供基础。

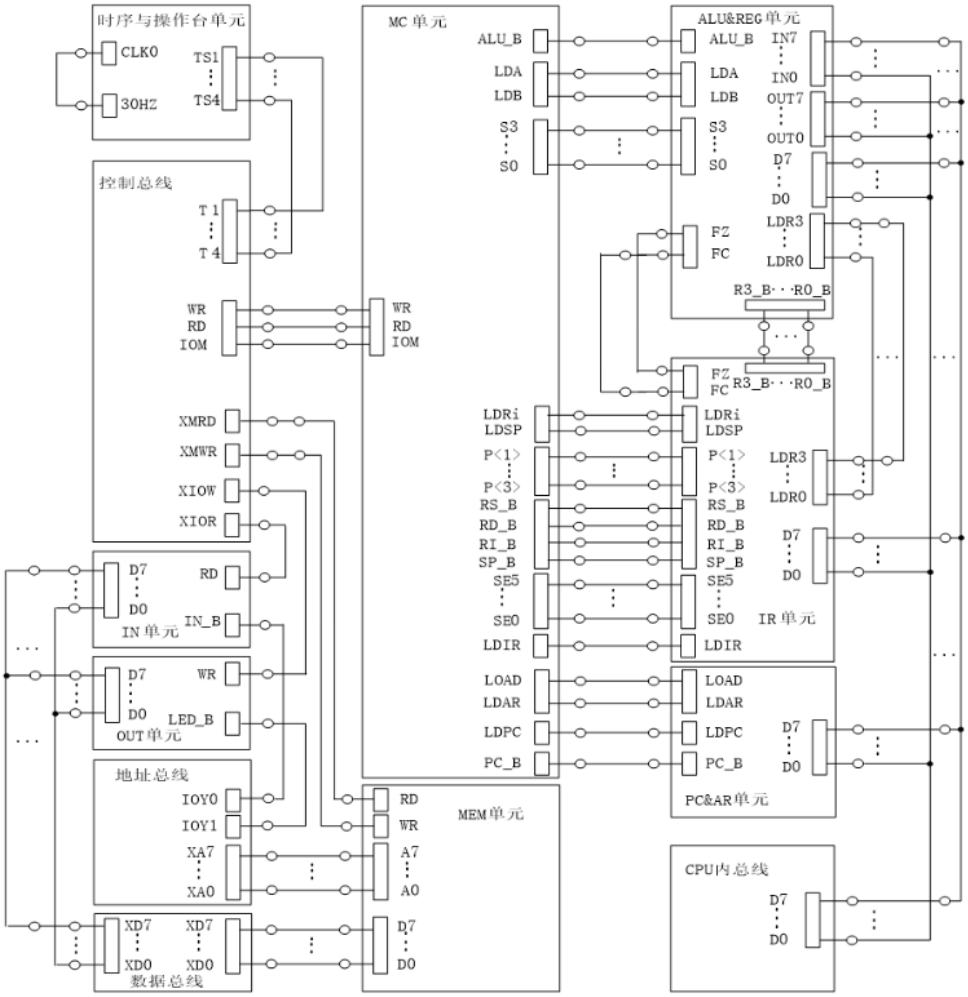


图 5-1 线路连线图

5.3 快速幂流程

1. 初始化寄存器：代码开始处，通过 IN 指令将输入值（基数 base、幂 power）分别加载到寄存器 R0、R1 中，并将这些 值存储到对应的内存地址（E0H、E1H、E2H）作为参数传递给快速幂函数。

2. 跳转到快速幂函数：通过 JMP QPOW 指令跳转到快速幂的实现部分。

3. 快速幂算法：

初始化返回值：将 ret（结果）初始化为 1，存储在寄存器 R3 中。

如果 power 大于 0，则循环计算：

如果 power 为奇数，则将当前 ret 与 base 相乘（模 p），并更新 ret。然后使用快乘实现：将 base 平方，并更新 base。将 power 右移一位，即除以 2，以处理下一个幂次 。

4. 存储结果并输出：将计算结果 ret 存储到内存地址 E3H 中，再将结果从 E3H 加载到寄存器 R2 中，并通过 OUT 指令输出，然后执行 HALT 指令结束程序。

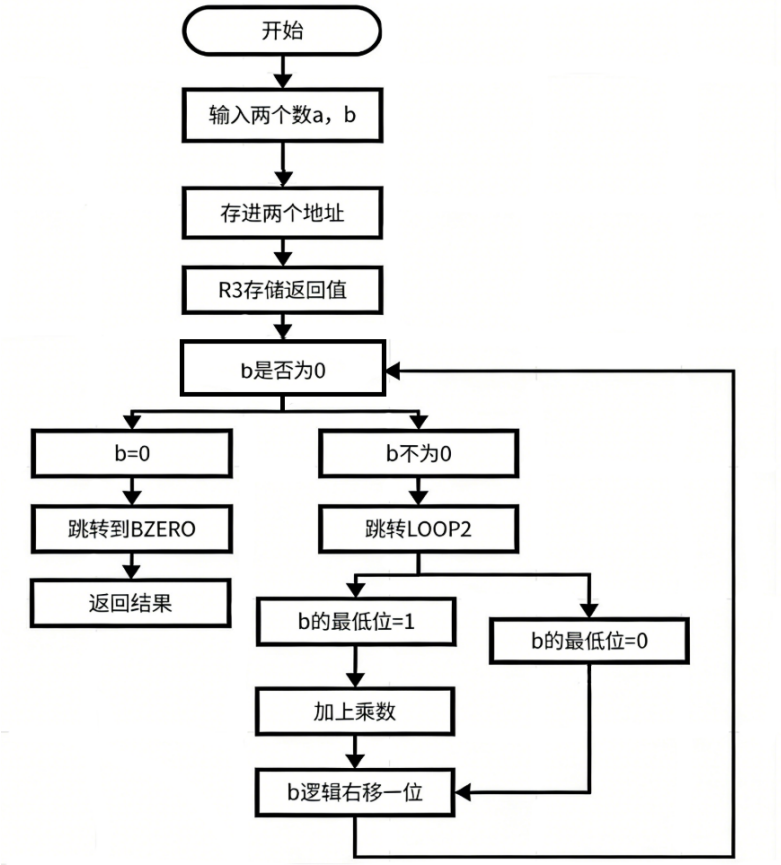


图 5-2 快速幂流程图

5.4 快乘流程

1. 输入参数

输入参数包括两个数 a 和 b，分别表示乘数和被乘数。这两个参数通过 STA 指令分别存储在内存地址 F0H 和 F1H 中。R3 用存储返回值 ret。

2. 初始化

LDI R3, 00H：初始化结果 ret 为零 。

LDI R2, 00H：检查 b 是否为零。如果 b 为零，直接跳转到 BZERO 标签，直接返回结果。

3. 二进制乘法主循环 (LOOP2)

LDI R2, 01H：检查当前位，使用 AND R2, R1 来判断 b 当前的最低位是否为 1。如果 b 当前最低位为 1（即 b%2 == 1），则将当前乘数 a 加到 ret 上：ADD R3, R0。

将乘数 a 自乘：MOV R2, R0 然后执行 ADD R0, R2，即 a = a + a，相当于将乘数加倍。接着将 b 右移一位，即 b /= 2：RR R1, R2。通过逻辑右移，确保 b 的每一位都被逐步处理。

4. 继续迭代

每次迭代之后，检查 b 是否为零，如果 b 为零则跳转到 BZERO 标签，表示乘法结束。如果 b 不为零，则继续迭代，继续处理 b 的下一位。

5. 结束与返回

BZERO 标签：当 b 为零时，结束乘法运算，保存结果并返回。STA F3H, R3 将最终结果保存到 F3H 中 。

根据之前的调用逻辑，QMUL 函数的结果会通过 STA 保存在 F3H 中，作为最终结果返回。

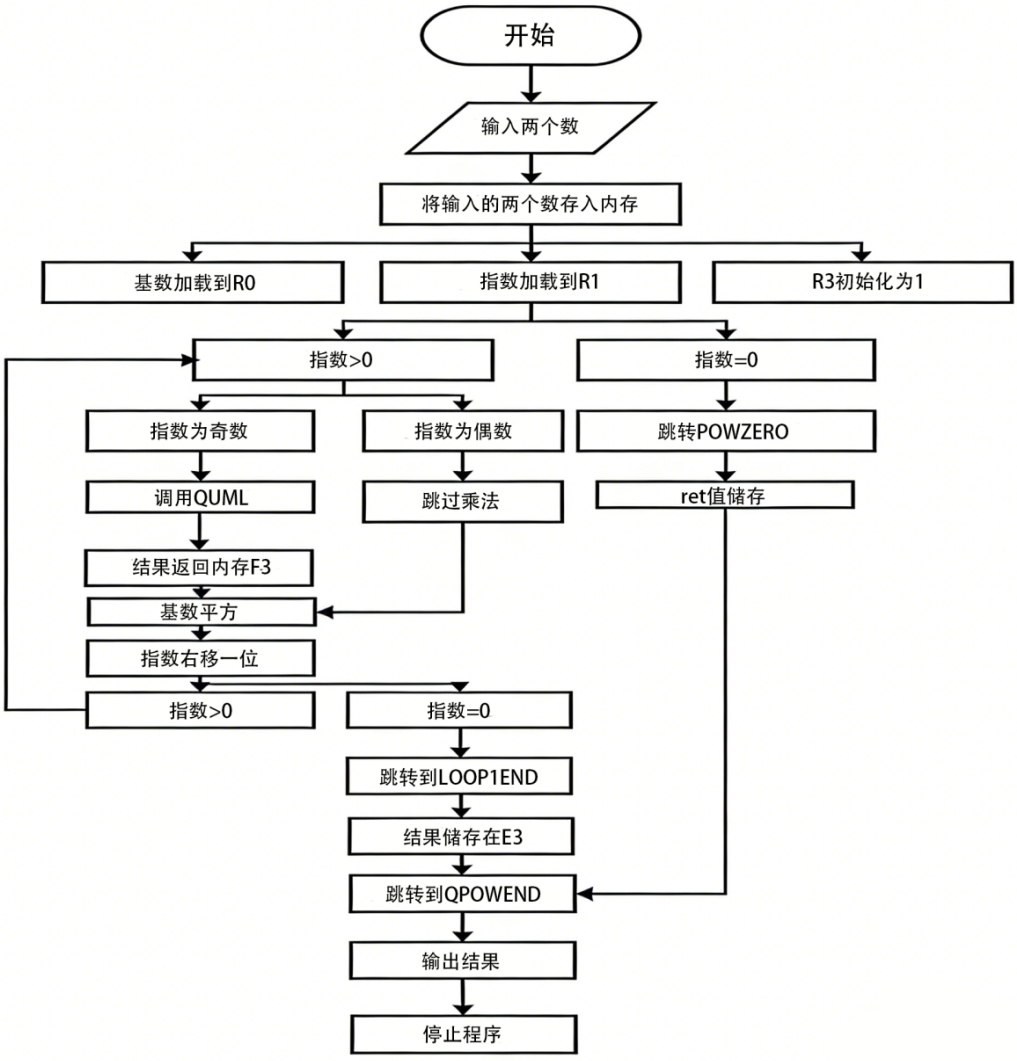

开始
输入两个数
将输入的两个数存入内存
基数加载到R0
指数加载到R1
R3初始化为1
指数>0
指数=0
指数为奇数
指数为偶数
跳转POWZERO
调用QUML
跳过乘法
ret值储存
结果返回内存F3
基数平方
指数右移一位
指数>0
指数=0
跳转到LOOP1END
结果储存在E3
跳转到QPOWEND
输出结果
停止程序

图 5-3 快速幂图

6 项目成果

6.1 线路连接

根据所给连线图对实验箱进行连线，如图 6-1：

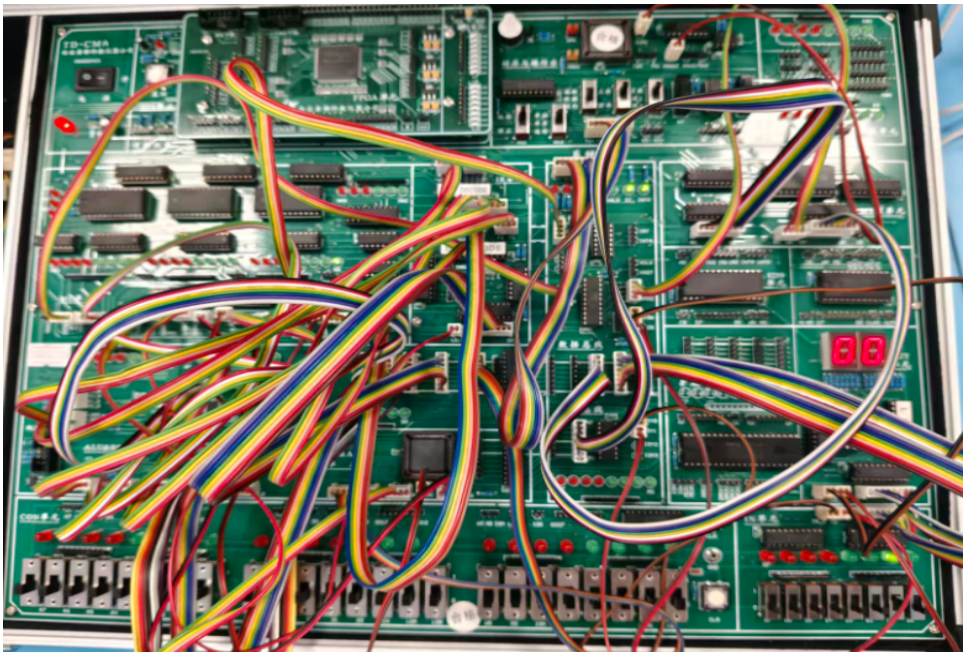


图 6-1 实验箱连线图

6.2 测试样例

输入样例：7 2（7 为底数 base，2 为指数 power，输入 7 后单周期运行，等待 base 传入寄存器 R0 后输入 2）

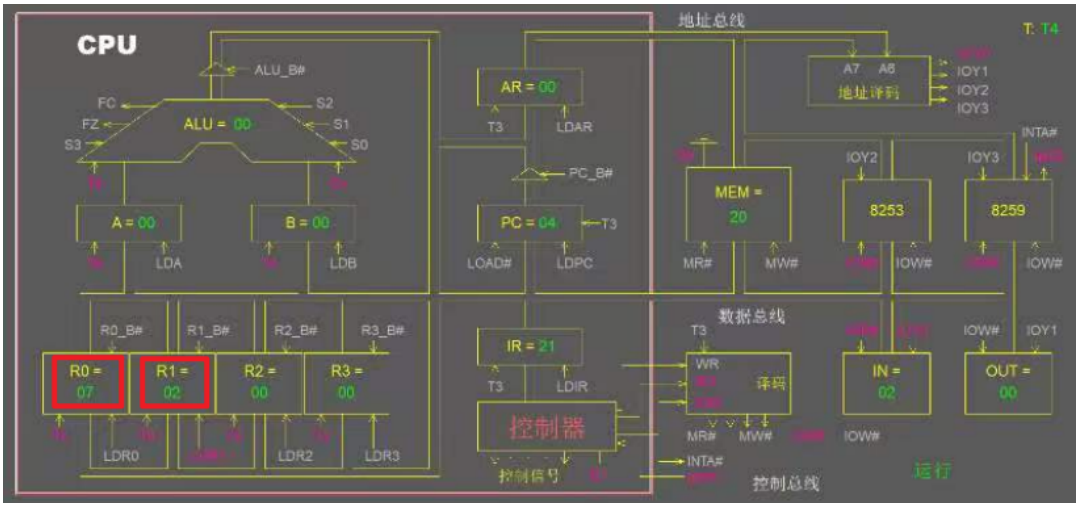


图 6-2 输入效果演示图

输出样例：31 （7 的 2 次方 = 49 = 31H，得验证算法正确）

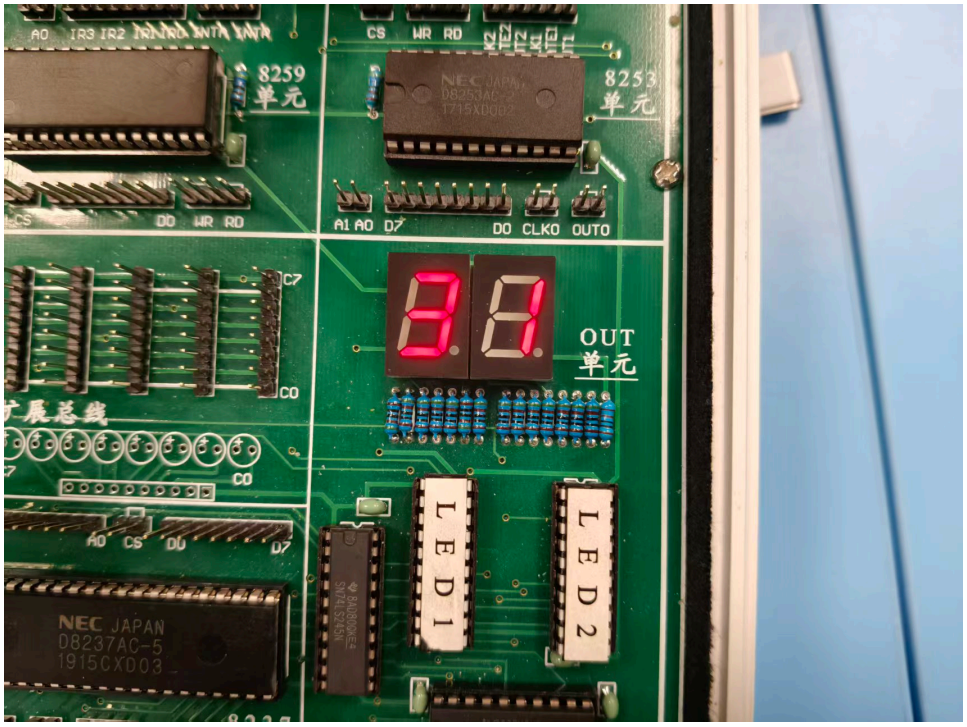


图 6-3 输出效果演示图

6.3 完整代码

完整汇编程序：见附录 A

完整机器码：见附录 B

完整 Python 转换程序：见附录 C

7 总结

本项目完成了“复杂模型机结合快速乘法实现快速幂”的设计与验证：在指令系统与微程序控制的支撑下，实现了快速幂与快速乘法的汇编化运行，并通过仿真与硬件连线运行验证了可行性与结果正确性，达成课程研究项目对组成原理理解与综合实践能力提升的目标。

参考文献：

[1]. 王爱英·计算机组成与结构（第 5 版）·清华大学出版社，2013。

[2]. 李涛·计算机组成原理实验教程·中国电力出版社，2009。

[3]. 李清泉·计算机组成原理与设计·人民邮电出版社，2020。

附录 A

```
;==================== 约定 ====================
; E0 base  E1 pow  E3 result  E4 保存 R3
; F0 a  F1 b  F2 返回选择(0->END1,1->END2)  F3 qmul 结果
; R3 作为 ret
;==============================================
        IN   R0,00H          ; base
        IN   R1,00H          ; pow
        STA  E0H,R0
        STA  E1H,R1
        JMP  QPOW
LABEL QPOWEND:
        LAD  R2,E3H
        OUT  40H,R2
        HALT
;-------------------- QPOW: 快速幂 --------------------
LABEL QPOW:
        LAD  R0,E0H          ; base
        LAD  R1,E1H          ; pow
        LDI  R3,01H          ; ret=1
        LDI  R2,FFH
```

```
        AND  R2,R1
        BZC  LOOP1END          ; pow==0
LABEL LOOP1:
        LDI  R2,01H
        AND  R2,R1
        BZC  UNMUL             ; if(!(pow&1)) skip
        ; ret = qmul(ret, base)
        STA  F0H,R3
        STA  F1H,R0
        LDI  R2,00H
        STA  F2H,R2
        STA  E0H,R0
        STA  E1H,R1
        JMP  QMUL
LABEL QMULEND1:
        LAD  R0,E0H
        LAD  R1,E1H
        LAD  R3,F3H            ; ret=F3
LABEL UNMUL:
        ; base = qmul(base, base)
        STA  F0H,R0
        STA  F1H,R0
        LDI  R2,01H
        STA  F2H,R2
        STA  E4H,R3
        STA  E1H,R1
```

```
        JMP  QMUL
LABEL QMULEND2:
        LAD  R1,E1H
        LAD  R3,E4H
        LAD  R0,F3H          ; base=F3
        ; pow >>= 1 (logical)
        LDI  R2,01H
        RR   R1,R2
        LDI  R2,7FH
        AND  R1,R2
        LDI  R2,FFH
        AND  R2,R1
        BZC  LOOP1END
        JMP  LOOP1
LABEL LOOP1END:
        STA  E3H,R3
        JMP  QPOWEND
;-------------------- QMUL: 移位加法乘 --------------------
LABEL QMUL:
        STA  E4H,R3          ; save R3
        LAD  R0,F0H          ; a
        LAD  R1,F1H          ; b
        LDI  R3,00H          ; ret=0
        LDI  R2,00H
        OR   R2,R1
        BZC  QMULRET         ; b==0
```

```
LABEL LOOP2:
        LDI   R2,01H
        AND   R2,R1
        BZC   NOADD
        ADD   R3,R0             ; if(b&1) ret+=a
LABEL NOADD:
        MOV   R2,R0
        ADD   R0,R2             ; a*=2
        LDI   R2,01H
        RR    R1,R2
        LDI   R2,7FH
        AND   R1,R2             ; b>>=1
        LDI   R2,00H
        OR    R2,R1
        BZC   QMULRET
        JMP   LOOP2
LABEL QMULRET:
        STA   F3H,R3            ; result
        LAD   R2,F2H
        LDI   R1,00H
        SUB   R2,R1
        BZC   QMULEND1
        JMP   QMULEND2
.ILM ilmilm.txt
```

附录 B

```
$P 00 20
```

```
$P 01 00
$P 02 21
$P 03 00
$P 04 D0
$P 05 E0
$P 06 D1
$P 07 E1
$P 08 E0
$P 09 0F
$P 0A C2
$P 0B E3
$P 0C 38
$P 0D 40
$P 0E 50
$P 0F C0
$P 10 E0
$P 11 C1
$P 12 E1
$P 13 63
$P 14 01
$P 15 62
$P 16 FF
$P 17 16
$P 18 F0
$P 19 56
$P 1A 62
```

```
$P 1B 01
$P 1C 16
$P 1D F0
$P 1E 35
$P 1F D3
$P 20 F0
$P 21 D0
$P 22 F1
$P 23 62
$P 24 00
$P 25 D2
$P 26 F2
$P 27 D0
$P 28 E0
$P 29 D1
$P 2A E1
$P 2B E0
$P 2C 5A
$P 2D C0
$P 2E E0
$P 2F C1
$P 30 E1
$P 31 62
$P 32 F2
$P 33 CB
$P 34 01
```

```
$P 35 D0
$P 36 F0
$P 37 D0
$P 38 F1
$P 39 62
$P 3A 01
$P 3B D2
$P 3C F2
$P 3D D3
$P 3E E4
$P 3F D1
$P 40 E1
$P 41 E0
$P 42 5A
$P 43 C1
$P 44 E1
$P 45 C3
$P 46 E4
$P 47 C0
$P 48 F3
$P 49 62
$P 4A 01
$P 4B A9
$P 4C 62
$P 4D 7F
$P 4E 19
```

```
$P 4F 62
$P 50 FF
$P 51 16
$P 52 F0
$P 53 56
$P 54 E0
$P 55 1A
$P 56 D3
$P 57 E3
$P 58 E0
$P 59 0A
$P 5A D3
$P 5B E4
$P 5C C0
$P 5D F0
$P 5E C1
$P 5F F1
$P 60 63
$P 61 00
$P 62 62
$P 63 00
$P 64 96
$P 65 F0
$P 66 7C
$P 67 62
$P 68 01
```

```
$P 69 16
$P 6A F0
$P 6B 6D
$P 6C 03
$P 6D 42
$P 6E 08
$P 6F 62
$P 70 01
$P 71 A9
$P 72 62
$P 73 7F
$P 74 19
$P 75 62
$P 76 00
$P 77 96
$P 78 F0
$P 79 87
$P 7A E0
$P 7B 67
$P 7C D3
$P 7D F3
$P 7E C2
$P 7F F2
$P 80 61
$P 81 00
$P 82 86
```

```
$P 83 F0
$P 84 2D
$P 85 E0
$P 86 43
$P 87 D3
$P 88 F3
$P 89 C2
$P 8A F2
$P 8B 61
$P 8C 00
$P 8D 86
$P 8E F0
$P 8F 2D
$P 90 E0
$P 91 43
$M 00 000001
$M 01 006D43
$M 03 107070
$M 04 002405
$M 05 04B201
$M 06 002407
$M 07 013201
$M 08 106009
$M 09 183001
$M 0A 106010
$M 0B 000001
```

```
$M 0C 103001
$M 0D 200601
$M 0E 005341
$M 0F 0000CB
$M 10 280401
$M 11 103001
$M 12 06B201
$M 13 002414
$M 14 05B201
$M 15 002416
$M 16 01B201
$M 17 002418
$M 18 02B201
$M 1B 005341
$M 1C 10101D
$M 1D 10608C
$M 1E 10601F
$M 1F 101020
$M 20 10608C
$M 28 101029
$M 29 00282A
$M 2A 04E22B
$M 2B 04928C
$M 2C 10102D
$M 2D 002C2E
$M 2E 04E22F
```

```
$M 2F 04928C
$M 30 001604
$M 31 001606
$M 32 006D48
$M 33 006D4A
$M 34 003401
$M 35 000035
$M 36 006D51
$M 37 001612
$M 38 001613
$M 39 001615
$M 3A 001617
$M 3B 000001
$M 3C 006D5C
$M 3D 006D5E
$M 3E 006D68
$M 3F 006D6C
```

附录 C

```
import re
import sys
from typing import Dict
OP = {
    "ADD": 0x0, "AND": 0x1,   "IN": 0x2,  "OUT": 0x3,   "STA": 0xD,
    "MOV": 0x4, "HALT": 0x5,  "LDI": 0x6, "INC": 0x7,   "JMP": 0xE,
    "SUB": 0x8, "OR": 0x9,    "RR": 0xA,  "LAD": 0xC,   "BZC": 0xF,
}
```

```
DEFAULT_MICRO: Dict[int, str] = {
    0x00: "000001",    0x01: "006D43",    0x03: "107070",
    0x04: "002405",    0x05: "04B201",    0x06: "002407",
    0x07: "013201",    0x08: "106009",    0x09: "183001",
    0x0A: "106010",    0x0B: "000001",    0x0C: "103001",
    0x0D: "200601",    0x0E: "005341",    0x0F: "0000CB",
    0x10: "280401",    0x11: "103001",    0x12: "06B201",
    0x13: "002414",    0x14: "05B201",    0x15: "002416",
    0x16: "01B201",    0x17: "002418",    0x18: "02B201",
    0x1B: "005341",    0x1C: "10101D",    0x1D: "10608C",
    0x1E: "10601F",    0x1F: "101020",    0x20: "10608C",
    0x28: "101029",    0x29: "00282A",    0x2A: "04E22B",
    0x2B: "04928C",    0x2C: "10102D",    0x2D: "002C2E",
    0x2E: "04E22F",    0x2F: "04928C",    0x30: "001604",
    0x31: "001606",    0x32: "006D48",    0x33: "006D4A",
    0x34: "003401",    0x35: "000035",    0x36: "006D51",
    0x37: "001612",    0x38: "001613",    0x39: "001615",
    0x3A: "001617",    0x3B: "000001",    0x3C: "006D5C",
    0x3D: "006D5E",    0x3E: "006D68",    0x3F: "006D6C",
}
class RegisterEnum:
    R0 = "00"
    R1 = "01"
    R2 = "10"
    R3 = "11"
    SP = R3
```

```
    RI = R2
    LUT = {"R0": R0, "R1": R1, "R2": R2, "R3": R3, "SP": SP, "RI":
RI}
class LocateEnum:
    IMMEDIATE = "00"
    MEMORY = "01"
    REGISTER_IMMEDIATE = "10"
    RELATIVE = "11"
def opcode_bits(name: str) -> str:
    return format(OP[name], "04b")
def encode_reg_to_reg(op_name: str, rd: str, rs: str) -> str:
    return (
        opcode_bits(op_name)
        + RegisterEnum.LUT[rs.upper()]
        + RegisterEnum.LUT[rd.upper()]
    )
def number_process(number: str) -> str:
    number = number.strip().upper()
    if number.endswith("H"):
        return bin(int(number[:-1], 16))[2:].zfill(8)
    if number.endswith("B"):
        return number[:-1].zfill(8)
    num = int(number)
    return bin((1 << 8) + num)[2:].zfill(8) if num < 0 else
bin(num)[2:].zfill(8)
def num_int(s: str) -> int:
```

```
    s = s.strip().upper()
    if s.endswith("H"):
        return int(s[:-1], 16)
    if s.endswith("B"):
        return int(s[:-1], 2)
    return int(s)
def clean(line: str) -> str:
    return line.split(";", 1)[0].strip()
def is_num(tok: str) -> bool:
    t = tok.strip().upper()
    return bool(re.fullmatch(r"-?\d+", t) or t.endswith("H") or
t.endswith("B"))
def strip_label(tok: str) -> str:
    return tok.strip().lstrip("!").strip().upper()
class Assembler:
    def __init__(self):
        self.machine_code = ["UNUSED"] * 1024
        self.byte_index = 0
        self.label_table = {}
    def put(self, bits: str, comment: str = ""):
        self.machine_code[self.byte_index] = f"{bits} ; {comment}"
if comment else bits
        self.byte_index += 1
    def _second_scan(self):
        for idx, line in enumerate(self.machine_code, start=1):
            if isinstance(line, str) and line.startswith("LABEL"):
```

```
                label = strip_label(line.replace("LABEL", "", 1))
                address = self.label_table[label]
                if idx - 2 >= 0 and "%M%" in self.machine_code[idx -
2]:
                    if address < 256:
                        self.machine_code[idx - 2] =
self.machine_code[idx - 2].replace(
                            "%M%", LocateEnum.IMMEDIATE
                        )
                        addr8 = number_process(str(address))
                    else:
                        self.machine_code[idx - 2] =
self.machine_code[idx - 2].replace(
                            "%M%", LocateEnum.RELATIVE
                        )
                        addr8 = number_process(str(address - idx))
                else:
                    addr8 = number_process(str(address))
                self.machine_code[idx - 1] = f"{addr8} ;LABEL
{label}"
def parse_mode_and_d(op: str):
    op = op.strip()
    if op.startswith("[") and op.endswith("]"):
        inner = op[1:-1].strip()
        if inner.upper().startswith("RI+"):
            d = inner.split("+", 1)[1].strip()
```

```
            return LocateEnum.REGISTER_IMMEDIATE, d
        return LocateEnum.MEMORY, inner
    return None, op
def asm_to_pm(lines):
    a = Assembler()
    for raw in lines:
        line = clean(raw)
        if not line:
            continue
        # LABEL xxx: / xxx:
        if line.upper().startswith("LABEL "):
            name = strip_label(line[6:].replace(":", ""))
            a.label_table[name] = a.byte_index
            continue
        if line.endswith(":"):
            name = strip_label(line[:-1])
            a.label_table[name] = a.byte_index
            continue
        up = line.upper()
        if up.startswith(".ORG"):
            a.byte_index = num_int(up.split(None, 1)[1])
            continue
        if up.startswith("DB"):
            v = up.split(None, 1)[1].strip()
            a.put(number_process(v), line)
            continue
```

```
        if up.startswith("LDI"):
            r, imm = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put("011000" + RegisterEnum.LUT[r.upper()], line)
            a.put(number_process(imm), "")
            continue
        if up.startswith("IN"):
            r, p = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put("001000" + RegisterEnum.LUT[r.upper()], line)
            a.put(number_process(p), "")
            continue
        if up.startswith("OUT"):
            p, r = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put("0011" + RegisterEnum.LUT[r.upper()] + "00", line)
            a.put(number_process(p), "")
            continue
        if up == "HALT":
            a.put(opcode_bits("HALT") + "0000", line)
            continue
        if up.startswith("ADD"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("ADD", rd, rs), line)
            continue
```

```
        if up.startswith("AND"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("AND", rd, rs), line)
            continue
        if up.startswith("OR"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("OR", rd, rs), line)
            continue
        if up.startswith("MOV"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("MOV", rd, rs), line)
            continue
        if up.startswith("SUB"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("SUB", rd, rs), line)
            continue
        if up.startswith("RR"):
            rd, rs = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            a.put(encode_reg_to_reg("RR", rd, rs), line)
            continue
        if up.startswith("GPC"):
```

```
            r = line.split(None, 1)[1].strip()
            a.put("010100" + RegisterEnum.LUT[r.upper()], line)
            continue
        if up.startswith("PUSH"):
            r = line.split(None, 1)[1].strip()
            a.put("1001" + RegisterEnum.LUT[r.upper()] + "00", line)
            continue
        if up.startswith("POP"):
            r = line.split(None, 1)[1].strip()
            a.put("101000" + RegisterEnum.LUT[r.upper()], line)
            continue
        if up == "STI":
            a.put("01110000", line)
            continue
        if up == "CLI":
            a.put("10000000", line)
            continue
        if up == "IRET":
            a.put("10110000", line)
            continue
        if up.startswith("LAD"):
            r, op = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            m, d = parse_mode_and_d(op)
            if m is None and not is_num(d):
```

```
                a.put("1100" + "%M%" + RegisterEnum.LUT[r.upper()],
line)
                a.put("LABEL " + strip_label(d), "")
            else:
                a.put(
                    "1100" + (m or LocateEnum.IMMEDIATE) +
RegisterEnum.LUT[r.upper()],
                    line,
                )
                a.put(
                    number_process(d) if is_num(d) else ("LABEL " +
strip_label(d)), ""
                )
            continue
        if up.startswith("STA"):
            op, r = [x.strip() for x in line.split(None,
1)[1].split(",", 1)]
            m, d = parse_mode_and_d(op)
            if m is None and not is_num(d):
                a.put("1101" + "%M%" + RegisterEnum.LUT[r.upper()],
line)
                a.put("LABEL " + strip_label(d), "")
            else:
                a.put(
                    "1101" + (m or LocateEnum.IMMEDIATE) +
RegisterEnum.LUT[r.upper()],
```

```
                    line,
                )
                a.put(
                    number_process(d) if is_num(d) else ("LABEL " +
strip_label(d)), ""
                )
            continue
        if up.startswith("JMP") or up.startswith("BZC"):
            op = line.split(None, 1)[1].strip()
            opcode = "1110" if up.startswith("JMP") else "1111"
            m, d = parse_mode_and_d(op)
            if m is None and not is_num(d):
                a.put(opcode + "%M%" + "00", line)
                a.put("LABEL " + strip_label(d), "")
            else:
                a.put(opcode + (m or LocateEnum.IMMEDIATE) + "00",
line)
                a.put(
                    number_process(d) if is_num(d) else ("LABEL " +
strip_label(d)), ""
                )
            continue
    a._second_scan()
    out = []
    for addr, cell in enumerate(a.machine_code):
        if cell == "UNUSED":
```

```
            continue
        bits = cell.split(";", 1)[0].strip()
        if bits.startswith("LABEL"):
            continue
        byte = int(bits, 2) & 0xFF
        out.append(f"$P {addr:02X} {byte:02X}")
    micro = [f"$M {addr:02X} {value}" for addr, value in
sorted(DEFAULT_MICRO.items())]
    out.extend(micro)
    return "\n".join(out) + "\n"
if __name__ == "__main__":
    inp, outp = sys.argv[1], sys.argv[2]
    with open(inp, "r", encoding="utf-8") as f:
        txt = f.readlines()
    with open(outp, "w", encoding="utf-8") as f:
        f.write(asm_to_pm(txt))
```